14

插叙：内存 API

在这个插叙中，我们讨论 UNIX 系统中的内存分配接口。这些接口非常简单，所以本章也短小精悍[1]。我们要解决的主要问题是：

> 关键问题：如何分配和管理内存
> 在 UNIX/C 程序中，理解如何分配和管理内存，是构建健壮、可靠软件的关键。常用的接口有哪些？应该避免哪些错误？

14.1 内存类型

C 程序运行时会分配两种内存。第一种叫作**栈**（stack）内存，它的分配和释放由编译器替你（程序员）隐式管理，所以有时也叫**自动**（automatic）内存。

在 C 语言里，要在栈上分配内存很容易。例如，假设你要在函数 `func()` 中为一个整数 `x` 分配一些空间，只需这样写：

```
void func() {
  int x; // 在栈上声明一个整数
  ...
}
```

剩下的事由编译器完成：你调用 `func()` 时，它会确保在栈上腾出空间；你从函数返回时，它会替你释放这块内存。因此，如果你希望某些信息在函数调用结束后依然存在，最好别把它放在栈上。

正是因为需要长期存在的内存，才有了第二种内存：**堆**（heap）内存。它的所有分配和释放都由你（程序员）显式处理。这无疑是一份沉重的责任！当然，这也是许多 bug 的根源。不过只要你足够细心，就能正确使用这些接口，不会遇到太多麻烦。下面这个例子在堆上分配了一个整数：

```
void func() {
  int *x = (int *) malloc(sizeof(int));
  ...
}
```

[1]说真的，我们希望每一章都是这样！只不过我们觉得这一章更短、更切中要害。

这段小代码有几点值得说明。首先，你可能注意到了，这一行里既有栈分配，也有堆分配：编译器看到你声明了指针（int *x），就知道要为一个整数指针腾出空间；随后，程序调用 malloc()，在堆上为一个整数申请空间；这个例程返回该整数的地址（成功时如此，失败时返回 NULL），地址再存放到栈上，供程序使用。

堆内存要显式管理，用法也更多样，因此无论对用户还是对系统，它都带来了更多挑战。本章接下来就重点讨论堆内存。

14.2 malloc() 调用

malloc() 调用非常简单：你传给它一个大小，请它在堆上分配这么多空间；它要么成功，返回指向新分配空间的指针，要么失败，返回 NULL[2]。

查看手册页（manual page）就能知道使用 malloc 需要做些什么；在命令行输入 man malloc，你会看到：

```
#include <stdlib.h>
...
void *malloc(size_t size);
```

从这些信息可以看出，要使用 malloc，你只需包含头文件 stdlib.h。实际上，你甚至都不需要这样做，因为所有 C 程序默认都会链接 C 库，而 C 库里就有 malloc() 的代码；加上这个头文件，只是让编译器能检查你调用 malloc() 的方式是否正确（例如，传入的参数个数和类型是否正确）。

malloc() 只有一个参数，类型是 size_t，表示你需要多少字节。不过，大多数程序员不会在这里直接写数字（比如 10）；实际上，这算不上好的写法。通常的做法是借助各种例程和宏。例如，要为一个双精度浮点数分配空间，只需这样写：

```
double *d = (double *) malloc(sizeof(double));
```

哇，好多 double 啊！这次调用 malloc() 时用了 sizeof() 运算符来申请大小合适的空间。在 C 中，sizeof() 通常被看作编译时运算符，也就是说，实际大小在编译时就已经知道，传给 malloc() 的参数会直接换成一个数字（这里是 8，即 double 的大小）。正因如此，应该把 sizeof() 看作运算符，而不是函数调用（函数调用发生在运行时）。

你也可以把变量名（而不只是类型）传给 sizeof()，但在某些情况下可能得不到想要的结果，所以要小心。例如，看看下面这段代码：

> 提示：拿不准时，就试一试
> 如果你拿不准正在用的某个例程或运算符到底如何工作，最好的办法就是亲手试一试，确认它的行为符合预期。读手册页或其他文档固然有用，但真正重要的是它实际怎样工作。写点代码测一测！这无疑是确保代码按你期望运行的最好办法。事实上，为了核实我们关于 sizeof() 的说法确实没错，我们就是这么做的！

[2]注意，C 中的 NULL 其实没什么特别的，通常只是一个表示零的宏，例如 #define NULL 0，有时是 #define NULL (void *)0。

```
int *x = malloc(10 * sizeof(int));
printf("%d\n", sizeof(x));
```

第一行为一个包含 10 个整数的数组申请了空间，这没什么问题。但在下一行使用 `sizeof()` 时，它返回的是一个很小的值，比如 4（32 位机器）或 8（64 位机器）。这是因为在这里，`sizeof()` 以为我们只是在问一个整数指针有多大，而不是我们动态分配了多少内存。不过，有时 `sizeof()` 的结果确实和你预想的一样：

```
int x[10];
printf("%d\n", sizeof(x));
```

这时，编译器有足够的静态信息，知道分配了 40 个字节。

另一个需要小心的地方是字符串。为字符串申请空间时，请使用这样的惯用写法：`malloc(strlen(s) + 1)`，它用函数 `strlen()` 获取字符串的长度，再加上 1，为字符串结束符留出空间。在这里使用 `sizeof()` 可能会带来麻烦。

你可能还注意到，`malloc()` 返回的是 `void` 类型的指针。这只是 C 语言传回地址的一种方式：把地址交给程序员，由程序员决定怎么用。程序员还会再帮一把，用上所谓的**强制类型转换**（cast）；在上面的例子中，程序员把 `malloc()` 的返回类型转换成了指向 `double` 的指针。强制类型转换其实没起什么实际作用，只是告诉编译器和其他可能在读你代码的程序员：“没错，我知道自己在干什么。”对 `malloc()` 的结果做强制类型转换，只是让人放心一点；程序的正确性并不需要它。

14.3 `free()` 调用

其实，分配内存是整件事里容易的部分；什么时候释放、怎么释放，甚至要不要释放，才是难点。要释放不再使用的堆内存，程序员只需调用 **`free()`**：

```
int *x = malloc(10 * sizeof(int));
...
free(x);
```

这个例程只接受一个参数，即 `malloc()` 返回的指针。你可能注意到了，用户并不传入所分配区域的大小，这个大小必须由内存分配库自己记录。

14.4 常见错误

使用 `malloc()` 和 `free()` 时，有不少常见错误。下面这些，是我们讲授本科操作系统课程时一再见到的。这些例子全都能编译、运行，编译器一声不吭；要写出正确的 C 程序，编译通过是必要条件，但远远不够，这一点你以后会体会到（往往是吃了苦头之后）。

事实上，正确管理内存一直是个大难题，以至于许多较新的语言都支持**自动内存管理**（automatic memory management）。在这类语言中，分配内存时你会调用类似 `malloc()` 的东西（通常是 **new** 或类似的操作，用来分配一个新对象），但从来不需要调用什么来释放空间；**垃圾回收器**（garbage collector）会运行起来，找出哪些内存你已经不再引用，然后替你释放。

提示：编译通过或能运行 ≠ 程序正确
程序编译通过了（!），甚至正确运行了一次或很多次，并不说明它就是正确的。可能是许多因素凑在一起，让你以为它能正常工作，可后来某些情况一变，它就不行了。学生常见的反应是说（或者喊）“可它以前是好的啊!”，然后怪罪编译器、操作系统、硬件，甚至（恕我们直言）教授。但问题通常就出在你以为它会在的地方——你的代码里。先动手调试，再去怪别的部件吧。

忘记分配内存

许多例程都要求你在调用之前先分配好内存。例如，例程 `strcpy(dst, src)` 把字符串从源指针复制到目标指针。可是一不小心，你就可能这样写：

```
char *src = "hello";
char *dst;          // 糟糕！未分配
strcpy(dst, src); // 段错误，程序挂掉
```

运行这段代码，很可能会导致**段错误**（segmentation fault）[3]，这是个花哨的说法，意思是：**你这个愚蠢的程序员，内存用错了，我很生气。**

这里正确的代码应该是这样的：

```
char *src = "hello";
char *dst = (char *) malloc(strlen(src) + 1);
strcpy(dst, src); // 正常工作
```

或者，你也可以使用 `strdup()`，让生活更轻松。更多信息请阅读 `strdup` 的手册页。

没有分配足够的内存

与此相关的一个错误是没有分配足够的内存，有时也叫**缓冲区溢出**（buffer overflow）。在上面的例子中，常见的错误是给目标缓冲区（buffer）留的空间差一点才够。

```
char *src = "hello";
char *dst = (char *) malloc(strlen(src)); // 太小了!
strcpy(dst, src); // 正常工作
```

奇怪的是，这个程序常常看起来运行正常，这取决于 malloc 的实现方式以及其他许多细节。有时，字符串复制会越过所分配空间的末尾多写一个字节，但这可能并无害处，也许只是覆盖了一个不再使用的变量。有时，这类溢出的危害极大；事实上，系统中许多安全漏洞正是由此而来 [W06]。还有些时候，malloc 库本来就多分配了一点空间，所以你的程序其实并没有乱写别的变量的值，运行得好好的。另外一些时候，程序确实会出错崩溃。于是我们又学到了宝贵的一课：程序正确运行过一次，并不意味着它就是正确的。

[3]这个名字听起来很玄乎，但你很快就会知道，为什么这种非法内存访问叫作段错误；如果这都不能激励你读下去，还有什么能呢?

忘记初始化分配的内存

这种错误是：你正确地调用了 `malloc()`，却忘了往新分配的数据类型里填值。千万别这样！如果真忘了，程序迟早会遇到**未初始化读**（uninitialized read），也就是从堆中读到一些值未知的数据。谁知道里面是什么呢? 运气好的话，读到的值（比如零）还能让程序照常工作；运气不好，读到的就是随机而有害的东西。

忘记释放内存

另一个常见错误叫作**内存泄漏**（memory leak），发生在你忘记释放内存的时候。对于长时间运行的应用程序或系统（比如操作系统本身），这是个大问题：内存一点点泄漏，最终会把内存耗尽，到那时就只能重启了。因此，一般来说，一块内存用完以后，你应该确保把它释放掉。注意，用带垃圾回收（garbage collection）的语言在这里也帮不上忙：只要你还持有某块内存的引用，任何垃圾回收器都不会释放它；所以即使在更现代的语言中，内存泄漏依然是个问题。

有些情况下，不调用 `free()` 似乎也说得过去。例如，你的程序生命周期很短，很快就会退出；这时进程（process）一结束，操作系统就会清理它分配的所有页（page），因此严格来说并不会发生内存泄漏。这样做确实“行得通”（参见第 6 页的补充），但这多半是个不该养成的坏习惯，所以选择这种做法时要慎重。从长远来看，作为程序员，你的目标之一是养成好习惯；其中一个习惯，就是清楚自己是怎样管理内存的，并且（在 C 这样的语言中）释放自己分配的内存块。就算不释放也能侥幸过关，最好也还是养成习惯，把你显式分配的每一个字节都释放掉。

在用完之前就释放内存

有时程序还没用完内存就把它释放了；这种错误叫作**悬空指针**（dangling pointer），你也猜得到，这同样不是好事。之后再使用它，可能会让程序崩溃，或者覆盖有效的内存（例如，你调用了 `free()`，随后又调用 `malloc()` 分配别的东西，而这次分配重新用上了那块被错误释放的内存）。

重复释放内存

程序有时还会把同一块内存释放不止一次，这叫作**双重释放**（double free）。这样做的结果是未定义的。可以想象，内存分配库可能会被搞糊涂，做出各种奇怪的事情；常见的结果是崩溃。

错误地调用 `free()`

我们要讨论的最后一个问题是错误地调用 `free()`。毕竟，`free()` 只希望你传给它之前从 `malloc()` 得到的某个指针。如果传入别的值，就可能（而且确实会）出坏事。因此，这种**无效释放**（invalid free）很危险，当然也应该避免。

补充：为什么进程退出后就不会泄漏内存

编写短期运行的程序时，你可能会用 `malloc()` 分配一些空间。程序运行完、马上要结束了：退出前有必要调用一大堆 `free()` 吗？不这样做似乎不对，但实际上不会有任何内存真正“丢失”。原因很简单：系统中其实有两个层次的内存管理。

第一层由操作系统负责：进程运行时，操作系统把内存分给它；进程退出（或以其他方式终止）时再收回来。第二层在每个进程内部，例如调用 `malloc()` 和 `free()` 时在堆内进行的管理。即使你没有调用 `free()`（从而泄漏了堆中的内存），程序运行结束时，操作系统也会回收这个进程的全部内存（包括代码页、栈页，以及这里涉及的堆页）。不管你地址空间（address space）中的堆是什么状态，进程结束时操作系统都会收回所有这些页，所以即使你没释放内存，也不会有内存丢失。

因此，对于短期运行的程序，泄漏内存通常不会造成什么实际问题（尽管这算不上好的写法）。但如果你写的是长时间运行的服务器（比如永不退出的 Web 服务器或数据库管理系统（database management system）），泄漏的内存就是大得多的问题，最终会在应用程序耗尽内存时导致崩溃。当然，有一个程序里的内存泄漏问题还要严重得多，那就是操作系统本身。这再次说明：写内核代码的人，干的是最难的活……

小结

如你所见，误用内存的方式有很多。正因为内存错误频繁出现，已经形成了一整套工具生态，帮你找出代码中的这类问题。去看看 **purify** [HJ92] 和 **valgrind** [SN05] 吧，两者都很擅长帮你找到内存相关问题的根源。一旦用惯了这些强大的工具，你会纳闷以前没有它们是怎么过来的。

14.5 底层操作系统支持

你可能注意到了，我们讨论 `malloc()` 和 `free()` 时一直没提系统调用（system call）。原因很简单：它们不是系统调用，而是库调用（library call）。malloc 库在你的虚拟地址空间（virtual address space）内管理空间，而它本身建立在一些系统调用之上，这些系统调用会进入操作系统，申请更多内存，或者把一些内存还给系统。

其中一个系统调用叫作 `brk`，用来改变程序 **break**（即 program break，常译作“程序间断点”）的位置，也就是堆末尾的位置。它接受一个参数（新 break 的地址），根据新 break 比当前 break 大还是小，相应地增大或缩小堆。另一个调用 `sbrk` 传入的是增量，其他方面用途类似。

注意，永远不要直接调用 `brk` 或 `sbrk`。它们是给内存分配库用的；你要是自己去用，很可能会出（大）乱子。还是老老实实用 `malloc()` 和 `free()` 吧。

最后，你还可以通过 `mmap()` 调用从操作系统获取内存。传入合适的参数，`mmap()` 就能在你的程序中创建一个**匿名**（anonymous）内存区域。这个区域不与任何特定文件关联，而是与**交换空间**（swap space）关联；交换空间我们稍后讲虚拟内存（virtual memory）时会详细讨论。这块内存同样可以当作堆来使用和管理。更多细节请阅读 `mmap()` 的手册页。

 © 2008–25, ARPACI-DUSSEAU

14.6 其他调用

内存分配库还支持其他一些调用。例如，`calloc()` 分配内存后还会在返回前把它清零；这样可以避免一类错误：你以为内存已经清零，于是忘了自己初始化（参见前面讲“未初始化读（uninitialized read）”的那一段）。如果你已经为某样东西（比如数组）分配了空间，后来又要往里添加内容，例程 `realloc()` 也很有用：它会创建一块更大的新内存区域，把旧区域的内容复制过去，并返回指向新区域的指针。

14.7 小结

本章介绍了一些与内存分配有关的 API。和往常一样，我们只讲了基础知识，更多细节可以看其他资料，比如《C 程序设计语言》[KR88] 和 Stevens 的《UNIX 环境高级编程》[SR05]（第 7 章）。如果想看一篇很酷的现代论文，了解如何自动检测并纠正其中许多问题，请参阅 Novark 等人的论文 [N+07]；这篇论文还很好地总结了常见问题，并提出了一些发现和修复这些问题的巧妙思路。

 © 2008–25, ARPACI-DUSSEAU

参考文献

[HJ92] "Purify: Fast Detection of Memory Leaks and Access Errors" by R. Hastings, B. Joyce. USENIX Winter '92. *The paper behind the cool Purify tool, now a commercial product.*

[KR88] "The C Programming Language" by Brian Kernighan, Dennis Ritchie. Prentice-Hall 1988. *The C book, by the developers of C. Read it once, do some programming, then read it again, and then keep it near your desk or wherever you program.*

[N+07] "Exterminator: Automatically Correcting Memory Errors with High Probability" by G. Novark, E. D. Berger, B. G. Zorn. PLDI 2007, San Diego, California. *A cool paper on finding and correcting memory errors automatically, and a great overview of many common errors in C and C++ programs. An extended version of this paper is available CACM (Volume 51, Issue 12, December 2008).*

[SN05] "Using Valgrind to Detect Undefined Value Errors with Bit-precision" by J. Seward, N. Nethercote. USENIX '05. *How to use valgrind to find certain types of errors.*

[SR05] "Advanced Programming in the UNIX Environment" by W. Richard Stevens, Stephen A. Rago. Addison-Wesley, 2005. *We've said it before, we'll say it again: read this book many times and use it as a reference whenever you are in doubt. The authors are always surprised at how each time they read something in this book, they learn something new, even after many years of C programming.*

[W06] "Survey on Buffer Overflow Attacks and Countermeasures" by T. Werthman. Available: www.nds.rub.de/lehre/seminar/SS06/Werthmann_BufferOverflow.pdf. *A nice survey of buffer overflows and some of the security problems they cause. Refers to many of the famous exploits.*

作业（编程）

这个作业会让你熟悉一下内存分配。首先，你要写几个有 bug 的程序（好玩!）。然后，用一些工具帮你找出自己埋下的 bug。接着，你会意识到这些工具有多棒，以后就会一直用它们，从而让自己更快乐、更高效。这些工具是调试器（例如 gdb）和名为 valgrind 的内存错误检测器 [SN05]。

问题

1. 首先，编写一个名为 null.c 的简单程序，它创建一个整数指针，将其设为 NULL，然后尝试对它解引用。把它编译成名为 null 的可执行文件。运行这个程序时会发生什么？
2. 接下来，编译这个程序时加上符号信息（使用 -g 选项）。这样可执行文件里会包含更多信息，调试器就能拿到变量名之类更有用的信息。输入 gdb ./null，在调试器中运行这个程序，等 gdb 运行起来后，再输入 run。gdb 显示了什么？
3. 最后，对这个程序使用 valgrind 工具。我们会用 valgrind 中的 memcheck 来分析发生了什么。输入以下命令运行：valgrind --leak-check=yes ./null。运行后发生了什么？你能看懂这个工具的输出吗？
4. 编写一个简单的程序，用 malloc() 分配内存，但在退出前忘记释放。这个程序运行时会发生什么？你能用 gdb 找出它的问题吗？用 valgrind（同样带上 --leak-check=yes 选项）呢？
5. 编写一个程序，用 malloc 创建一个名为 data、大小为 100 的整数数组；然后把 data[100] 设为零。运行这个程序时会发生什么？用 valgrind 运行这个程序时会发生什么？这个程序正确吗？
6. 编写一个程序，分配一个整数数组（同上），把它释放掉，然后尝试打印数组中某个元素的值。程序能运行吗？对它使用 valgrind 时会发生什么？
7. 现在给 free 传一个奇怪的值（例如，指向你上面分配的数组中间的指针）。会发生什么？你需要借助工具来发现这类问题吗？
8. 试试其他一些内存分配接口。例如，实现一个类似向量（vector）的简单数据结构（data structure），以及用 realloc() 管理这个向量的相关例程。用一个数组存储向量的元素；用户向向量添加一项时，用 realloc() 为它分配更多空间。这样的向量性能如何？和链表（linked list）相比怎么样？用 valgrind 帮你找 bug。
9. 多花些时间，读一读 gdb 和 valgrind 的用法。熟悉自己的工具至关重要；花点时间，让自己成为 UNIX 和 C 环境下的调试高手。